

Capture the Flag 5: Make the System Interactive

Goal: Transform your Student Directory into an interactive application where administrators can favorite students, edit information, and remove records using Flutter interactions and state management.

Instructor Note

These activities require your own problem-solving and research skills.

DO NOT use AI to generate the code for you.

You may use AI to:

- Research what a Dart/Flutter feature does
- Understand syntax
- Explain an error (not fix your error)
- Clarify documentation

You may NOT ask AI to generate the solution/code for a flag.

Flag 0 — Create Your Feature Branch

Objective: Create a new feature branch before adding interactions.

Create:

`feature/student-interactions`

Screenshot:

- Terminal showing your current branch

```
C:\Windows\System32\cmd.exe x + v
C:\Users\Lenovo\Desktop\IT314AB_Sucgang>git checkout -b feature/student-interactions
Switched to a new branch 'feature/student-interactions'

C:\Users\Lenovo\Desktop\IT314AB_Sucgang>git status
On branch feature/student-interactions
Changes to be committed:
  (use "git restore --staged <file> ..." to unstage)
        modified:   my_first_flutter_app/lib/data/student_data.dart
        modified:   my_first_flutter_app/pubspec.yaml

C:\Users\Lenovo\Desktop\IT314AB_Sucgang>
```

🚩 Flag 1 — Wake Up the Buttons

Add at least two buttons inside each Student Card.

Example actions:

- Favorite
- Edit

Objective: Make your Student Card respond to button presses using `onPressed`.

Right now your buttons are only decorations.

Research how Flutter's `onPressed` works.

The buttons do not need to perform their final behavior yet. They simply need to respond when pressed.

Test

When a button is pressed, your app should visibly react in some way (such as printing a message or showing a temporary notification).

Screenshot:

- Student Card with buttons

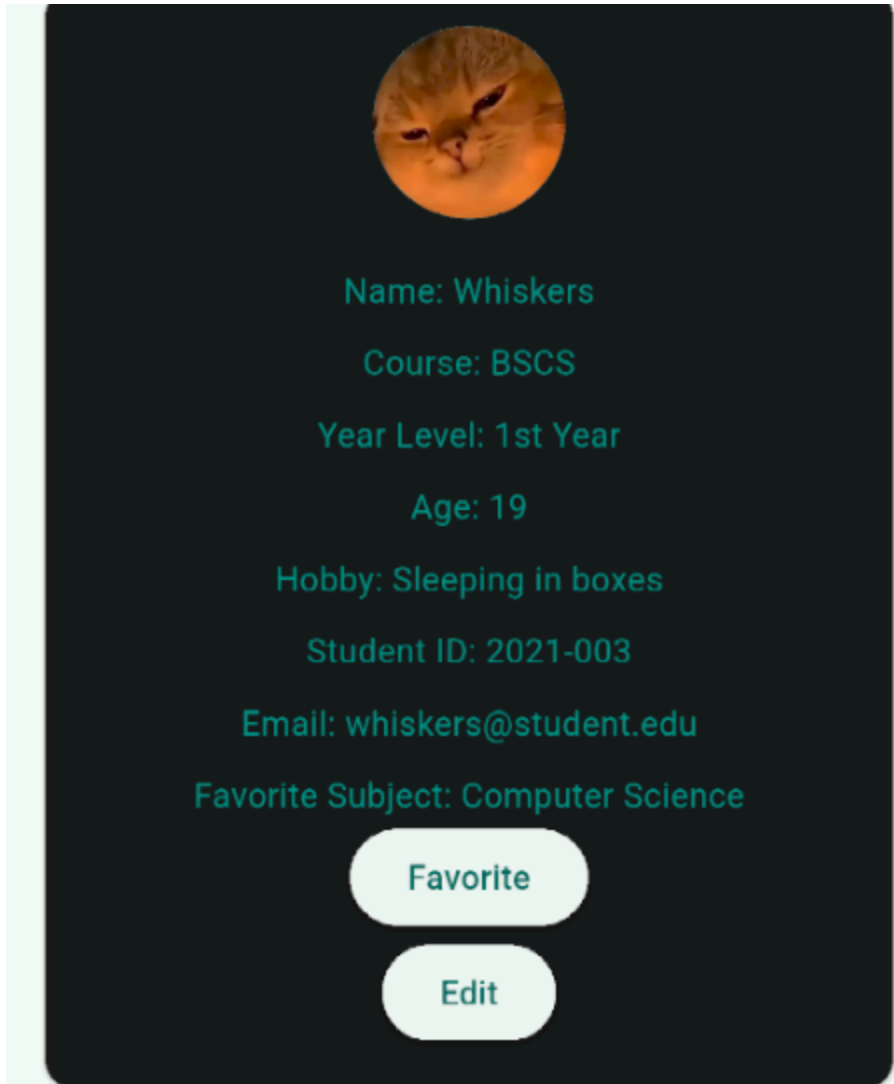
- `onPressed` inside your code
- Button reacting or terminal message

```
62 ElevatedButton(  
63   onPressed: () {  
64     print("Favorite"); Don't invoke 'print' in production code. Try using a logging framework.  
65   },  
66   child: Text("Favorite"),  
67 ), // ElevatedButton  
68 ElevatedButton(  
69   onPressed: () {  
70     print("Edit"); Don't invoke 'print' in production code. Try using a logging framework.  
71   },  
72   child: Text("Edit"),  
73 ), // ElevatedButton  
74 ],  
75 ), // Column  
76 ), // Padding  
77 ); // Card  
78 }  
79
```

dart - my_first_flutter_app

, GpuCompleted=531150160400, SwapBuffersCompleted=531190718800, DisplayPresentTime=129490261343840, CommandSubmissionComple
ed=531121536700,
D/WindowLayoutComponentImpl(5553): Register WindowLayoutInfoListener on Context=com.example.my_first_flutter_app.MainActivi
ty@c867b2b, of which baseContext=android.app.ContextImpl@3c02b59
D/FlutterJNI(5553): Sending viewport metrics to the engine.
D/InsetsController(5553): hide(ime())
I/ImeTracker(5553): com.example.my_first_flutter_app:325e48f2: onCancelled at PHASE_CLIENT_ALREADY_HIDDEN

Performing hot reload...
Reloaded 1 of 757 libraries in 2,230ms (compile: 77 ms, reload: 423 ms, reassemble: 537 ms).
I/flutter (5553): Favorite
I/flutter (5553): Edit
█



🚩 Flag 2 — Touch Anywhere

Objective: Make the entire Student Card tappable using `onTap`.

Buttons aren't the only interactive elements.

Research Flutter's `onTap`.

Wrap your Student Card so tapping anywhere on the card triggers an interaction.

Test

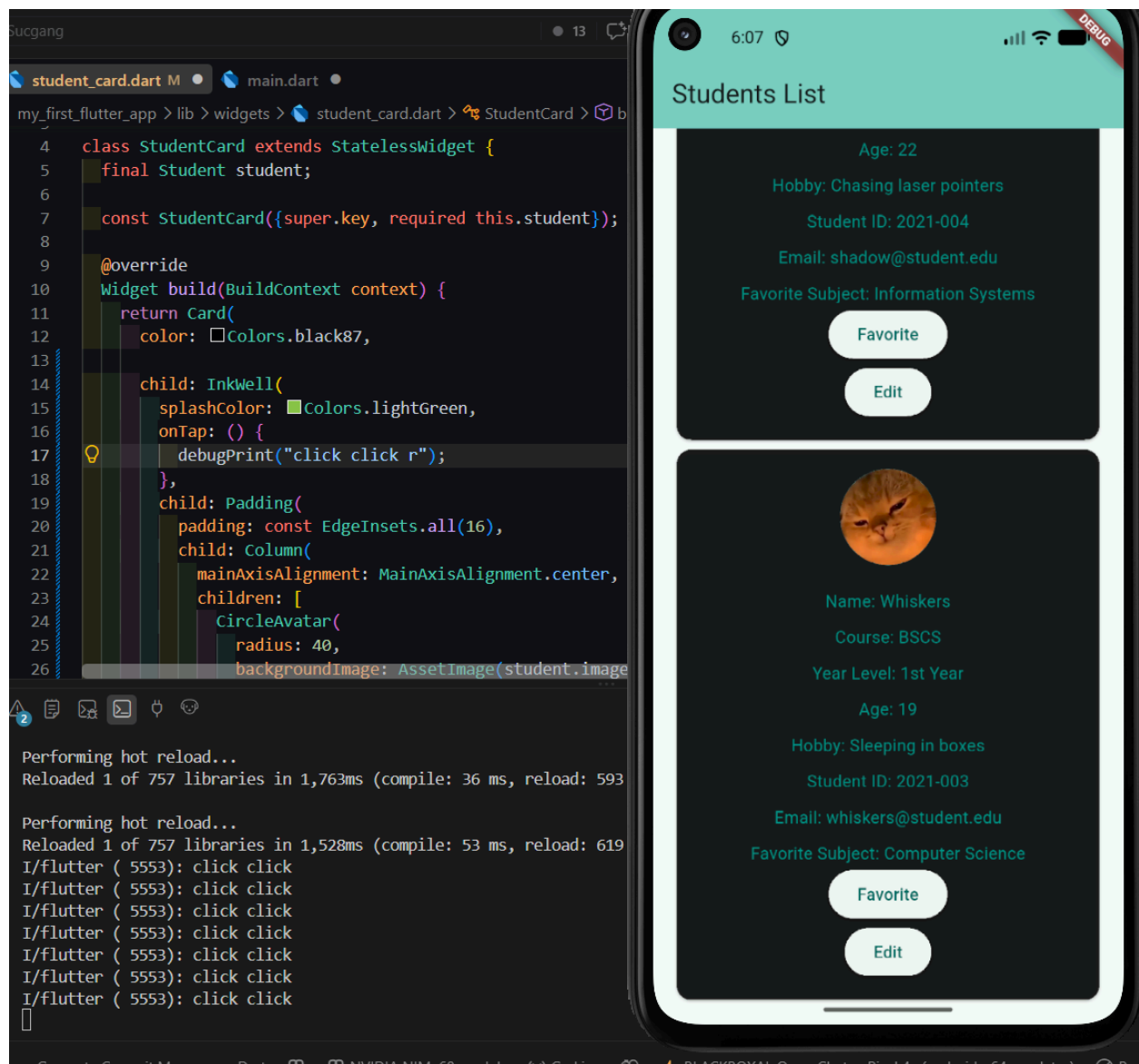
The user should be able to:

- Tap the Favorite button
- Tap the Edit button
- Tap anywhere else on the Student Card

Each interaction should be recognized separately. (One function for the 2 buttons, another for the Student Card)

Screenshot:

- `onTap` in your code
- Student Card being tapped



🚩 Flag 3 — Discover `setState`

Objective: Update the screen immediately using `setState`.

So far your app reacts. But does the UI actually change?

Research Flutter's `setState`.

Use it so pressing a button immediately updates something visible on the screen.

Examples include:

- Changing text
- Changing an icon
- Updating a counter

The important part is understanding that `setState` tells Flutter to rebuild the UI.

Test

The screen should visibly change immediately after pressing the button.

Screenshot:

- `setState` in your code
- Before and after interaction

```
ElevatedButton(  
  onPressed: () {  
    setState(() {  
      widget.student.age++;  
    });  
  },  
  child: Icon(Icons.plus_one),  
), // ElevatedButton  
],  
// Column  
// Padding  
Inkwell
```

Name: John the Cat

Course: BSIT

Year Level: 3rd Year

Age: 23

Hobby: Watching football

Student ID: 2021-001

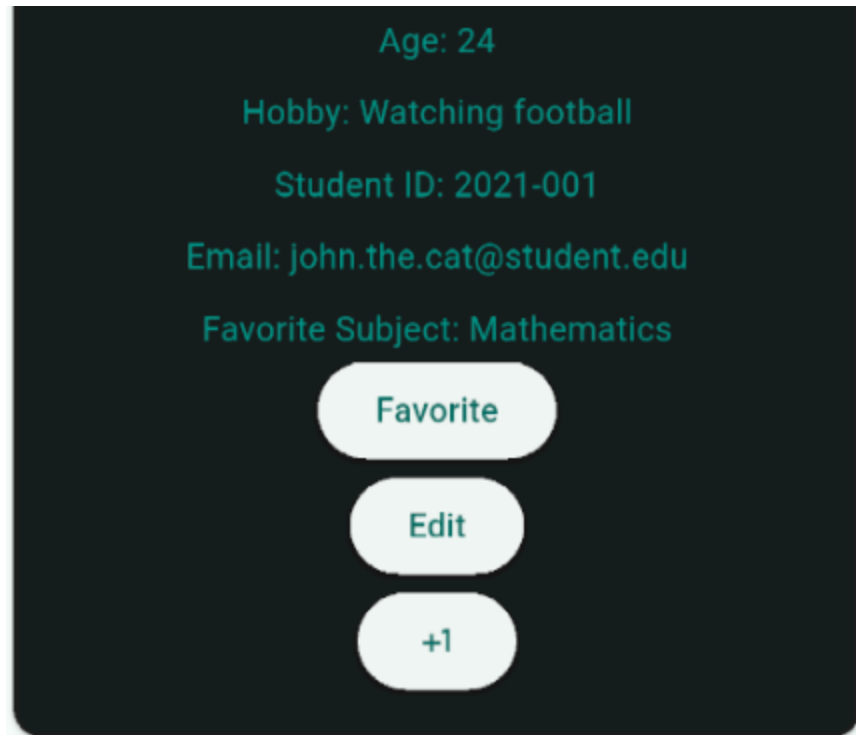
Email: john.the.cat@student.edu

Favorite Subject: Mathematics

Favorite

Edit

+1



🚩 Flag 4 — Favorite Students

Objective: Let administrators mark students as favorites.

Schools often need to highlight important records.

Add a favorite feature that changes a student's appearance when marked.

Possible visual changes include:

- Filled heart/star
- Different card color
- Favorite label

Each student's favorite status should work independently.

Test

- Favorite one student.
- Other students should remain unchanged.
- Favorite another student.

Both should keep their own state.

Screenshot:

- Favorited student
- Non-favorited student
- Favorite icon changing



Name: John the Cat
Course: BSIT
Year Level: 3rd Year
Age: 21
Hobby: Watching football
Student ID: 2021-001
Email: john.the.cat@student.edu
Favorite Subject: Mathematics

Favorite

Edit

+1



Name: Milo
Course: BSIT
Year Level: 3rd Year
Age: 21
Hobby: Bird watching
Student ID: 2021-005
Email: milo@student.edu
Favorite Subject: Mathematics

Favorite

Edit

+1



Name: Milo
Course: BSIT
Year Level: 3rd Year
Age: 21
Hobby: Bird watching
Student ID: 2021-005
Email: milo@student.edu
Favorite Subject: Mathematics

Favorite

Edit

+1

```
ElevatedButton(  
  onPressed: () {  
    setState(() {  
      widget.student.isFavorite = true;  
    });  
  },  
  child: Text("Favorite"),  
) , // ElevatedButton
```

```
if (widget.student.isFavorite == true)  
  Icon(Icons.favorite, color: Colors.red),
```

🚩 Flag 5 — Remove a Student

Objective: Delete a student from the directory.

Research how to remove an item from a Dart `List`.

Add a Delete action that removes a student from your list.

The UI should immediately update after deletion.

Test

Before:

- Student A
- Student B
- Student C

After deleting Student B:

- Student A
- Student C

No empty spaces should remain.

Screenshot:

- Student before deletion

- Student after deletion

Students List



Name: John the Cat
Course: BSIT
Year Level: 3rd Year
Age: 21
Hobby: Watching football
Student ID: 2021-001
Email: john.the.cat@student.edu
Favorite Subject: Mathematics

Favorite

Edit

+1

Delete



Name: Milo
Course: BSIT

Students List



Name: Milo
Course: BSIT
Year Level: 3rd Year
Age: 21
Hobby: Bird watching
Student ID: 2021-005
Email: milo@student.edu
Favorite Subject: Mathematics

Favorite

Edit

+1

Delete



Name: Mittens

🚩 Flag 6 — Edit Trigger

Objective: Prepare your app for editing student information.

In the next CTF you'll build a full Edit Screen.

For now, create an Edit trigger.

Research simple ways Flutter can display temporary editing interfaces.

Examples include:

- `AlertDialog`
- `SnackBar`
- Bottom Sheet

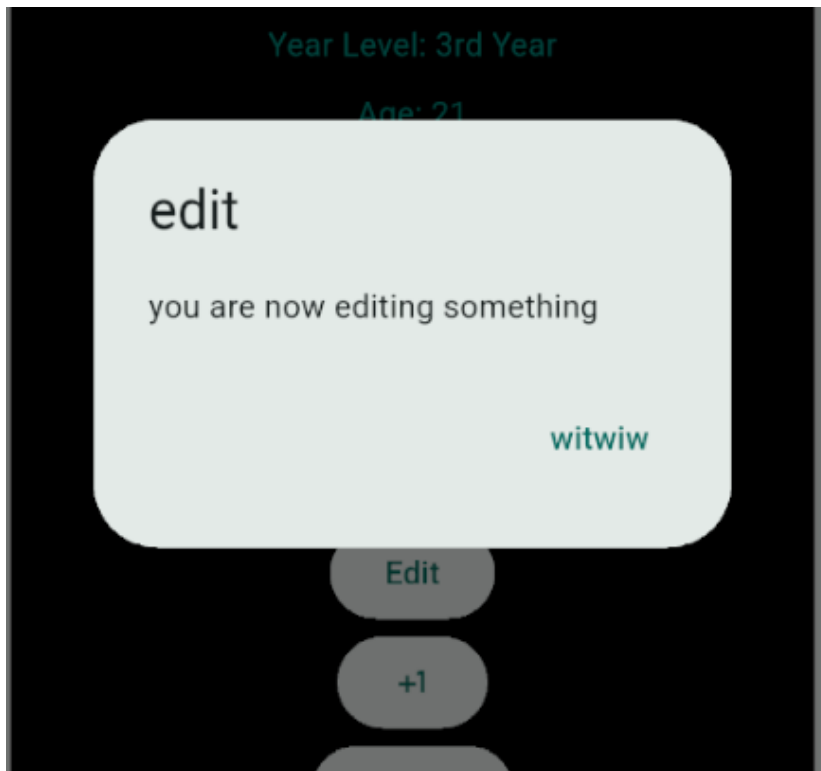
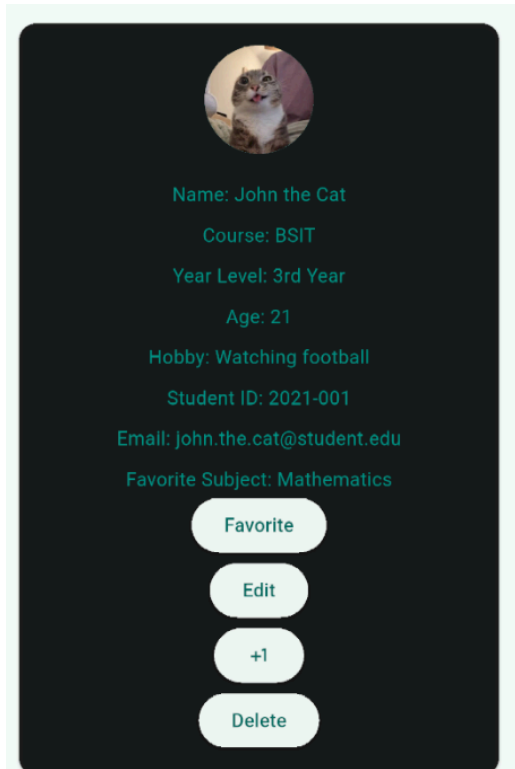
The goal is simply to launch an edit action, not build the complete editor yet.

Test

Pressing Edit should open your chosen interface.

Screenshot:

- Edit button
- Opened dialog, snackbar, or bottom sheet



Flag 7 — Multiple Actions, Independent State

Objective: Make every Student Card manage interactions correctly.

This is your final interaction challenge.

Verify that every student behaves independently.

Example scenario:

- Favorite Student A.
- Delete Student C.
- Open Edit for Student B.

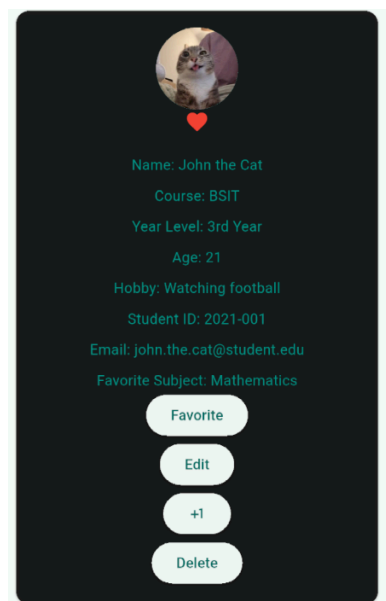
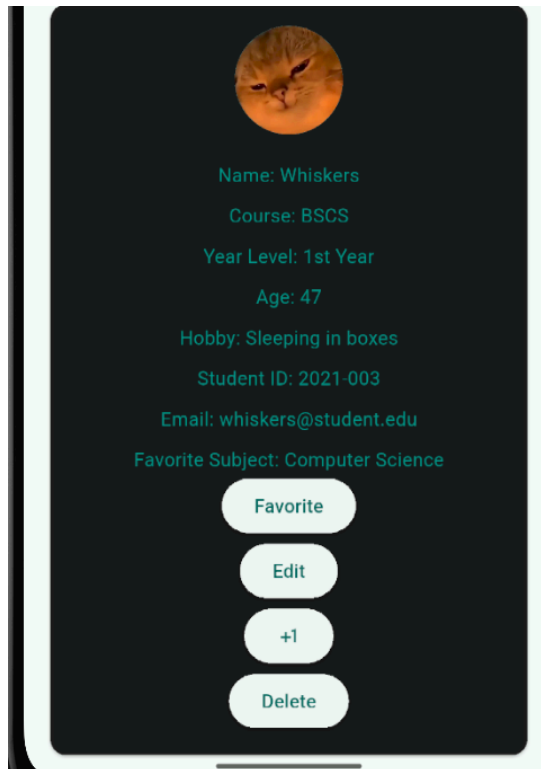
Your application should continue functioning correctly without affecting unrelated students.

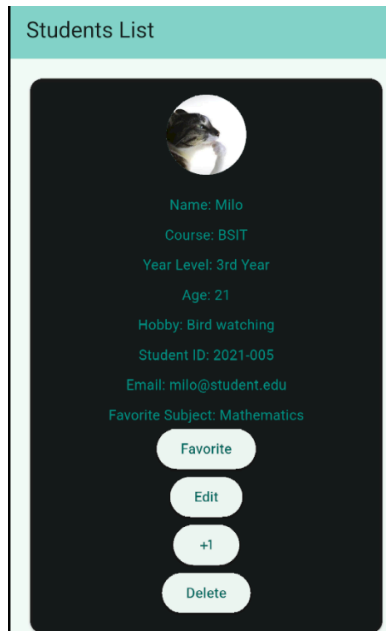
Test Checklist

- Favorite works independently.
- Delete updates the list immediately.
- Edit still opens correctly.
- Remaining students keep their own states.

Screenshot:

- Multiple interactions working together
- Application still functioning normally





EVERYTHING IS STILL FUNCTIONAL

🚩 Flag 8 — Save Your Progress

Objective: Commit and push your completed Interactive Student Directory.

Use a proper Git commit message.

Branch:

`feature/student-interactions`

Screenshot:

- Successful commit
- Current branch
- Terminal showing the commit


```
C:\Users\Lenovo\Desktop\IT314AB_Sucgang>git push -u origin feature/student-interactions
Enumerating objects: 23, done.
Counting objects: 100% (23/23), done.
Delta compression using up to 12 threads
Compressing objects: 100% (10/10), done.
Writing objects: 100% (12/12), 2.14 KiB | 729.00 KiB/s, done.
Total 12 (delta 6), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (6/6), completed with 6 local objects.
To https://github.com/Eziki43/IT314AB_Sucgang.git
   fb9f26a..22a7149  feature/student-interactions -> feature/student-interactions
branch 'feature/student-interactions' set up to track 'origin/feature/student-interactions'.

C:\Users\Lenovo\Desktop\IT314AB_Sucgang>git push
Everything up-to-date

C:\Users\Lenovo\Desktop\IT314AB_Sucgang>git branch -a
  Capture-The-Flag
+ Capture_The_Flag
  feature/data-ui
* feature/student-interactions
  feature/student-list
  feature/widgets
  main
remotes/origin/Capture-The-Flag
remotes/origin/HEAD -> origin/main
remotes/origin/feature/data-ui
remotes/origin/feature/student-interactions
remotes/origin/feature/student-list
remotes/origin/feature/widgets
remotes/origin/main

C:\Users\Lenovo\Desktop\IT314AB_Sucgang>
```